

1. Using Single Perceptron to Identify Patterns:

A. an example of binary classification, the perceptron used to classify data points as belonging to two classes and the classes are linearly separable.

we can use the perceptron above to identify specific patterns.

in this case, the perceptron above classifies inputs as either "belonging to the class of specific pattern" or not "belonging to the class of specific pattern",

we are still doing binary classification, so the one is specific pattern, but the other can only "everything else".

B. there are still many patterns whose classes are **not** linearly separable, thus in order to identify such patterns (Ex: XOR problem), we need to combine multiple neurons (perceptrons) into a neural network to solve them.

2. Example: Training a Single Perceptron to Identify Target Pattern:

A. Step1. Target Pattern :

Input(x): 9 pixels. White = +1.0, Black = -1.0

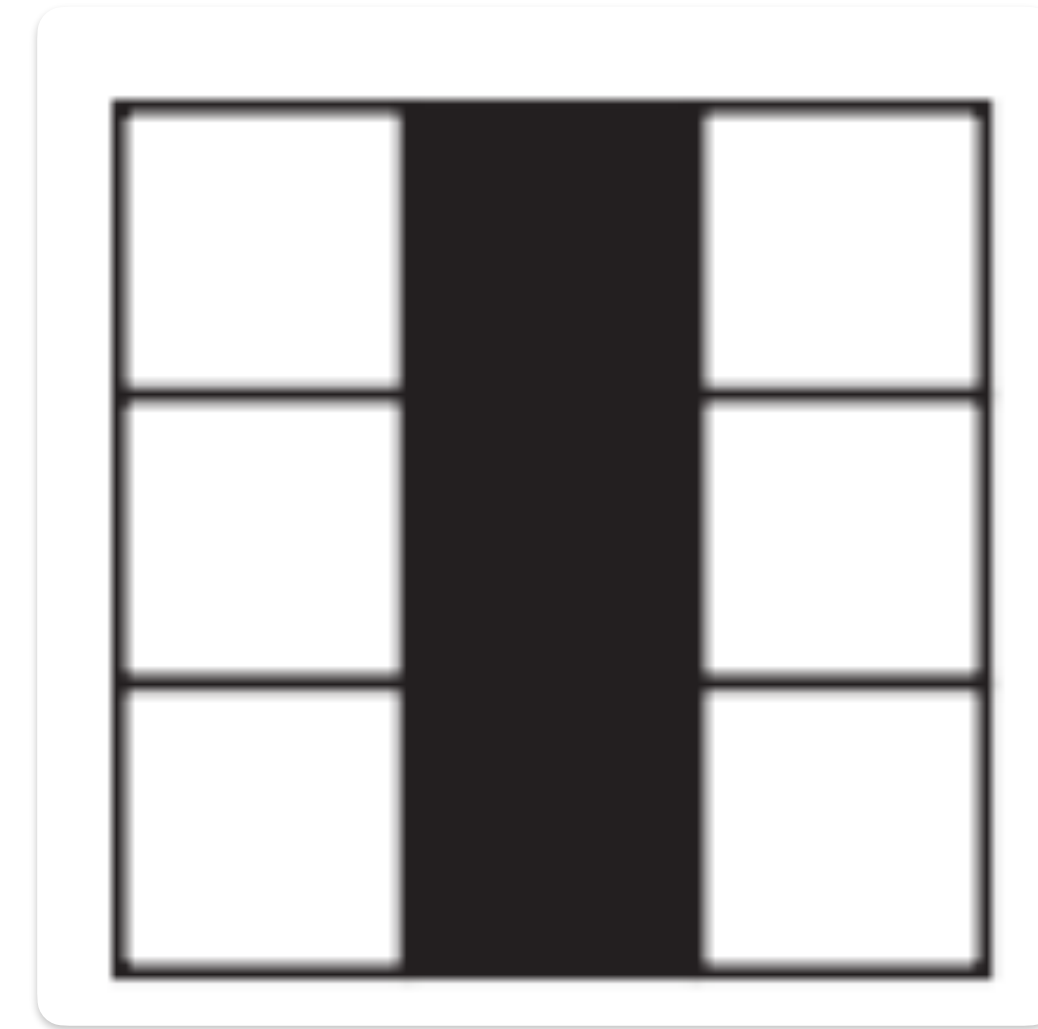
Ground Truth (y) = +1

Target x Matrix:

[x1= +1, x2= -1, x3= +1]

[x4= +1, x5= -1, x6= +1]

[x7= +1, x8= -1, x9= +1] x0= +1



Step2. Initial State :

Weights (w0~w9) are randomly initialized and inaccurate.

Predicted output (p_y) = sign(z).

Step3. Training Target Pattern :

Condition:

Input(x): 9 pixels. White = +1.0, Black = -1.0 for target pattern.

Ground Truth (y) = +1, but predicted output p_y = -1

Update Rule: $W = W + y * \text{learning rate} * X$

Step4. Training Non-Target Pattern :

Condition:

Input(x): 9 pixels. White = +1.0, Black = -1.0 for non-target pattern.

Ground Truth (y) = -1, but predicted output p_y = +1

Update Rule: $W = W + y * \text{learning rate} * X$

Step5. Final Convergence :

Finally adjusted Weight Matrix :

[w1= 0.2, w2= -0.2, w3= 0.8]

[w4= 0.8, w5= -0.2, w6= 0.4]

[w7= 0.4, w8= -0.4, w9= 0.2], w0= -3.4

Testing Target Pattern with finally adjusted weights :

$z = 0.2*1 + -0.2*-1 + 0.8*1 + 0.8*1 + -0.2*-1 + 0.4*1 + 0.4*1 + -0.4*-1 + 0.2*1 > 0 \rightarrow p_y = +1$

Predicted output (p_y) = +1 Ground Truth (y) = +1 --> Yes

B. looking at successfully classified target pattern,

if a pixel of target pattern is white (x=+1), the corresponding weight will finally be adjusted as a **positive** value so as to get higher value of z and make p_y matching y.

if a pixel of target pattern is black (x=-1), the corresponding weight will finally be adjusted as a **negative** value so as to get higher value of z and make p_y matching y.

C. even if successfully classified target pattern is perfectly symmetric, the finally adjusted weights matrix is not necessarily symmetric.

this is a result of the random initialization of weights and the perceptron learning algorithm just find out one of workable solutions (there are many workable finally adjusted weights matrix)